

Financial Agent

Decision-support platform for an investment fund managing a mixed portfolio of equities and cryptocurrencies.

The goal of this project is to turn raw market data into **actionable portfolio intelligence**. Instead of only showing price charts, the system highlights:

- how each asset performed
- how risky it has been
- how much it tends to move with the market
- what action may make sense next

What We Built

The project is structured around three decision layers:

1. Portfolio Intelligence

For each asset, we calculate:

- Total Return over 30D, 1Y, and 5Y
- Annualized Volatility
- Max Drawdown
- Sharpe Ratio
- Correlation with SPY
- Risk Score
- Opportunity Score
- Asset Rank

This gives a clear picture of performance, downside risk, and diversification value.

2. Actionable Recommendation

Each asset receives:

- buy

- hold
- reduce

The recommendation is backed by:

- confidence
- risk_level
- rationale
- key_drivers

This makes the output explainable and useful for portfolio review.

3. Heuristic Projection

Instead of pretending to predict an exact future price, we use a **short-term heuristic projection**.

The projection is based on:

- 30-day momentum
- 90-day momentum
- volatility

It returns:

- trend_outlook: bullish, neutral, or bearish
- projection_confidence
- projected_range: low, mid, high
- scenario_note

This is intended as a **scenario view**, not a guaranteed forecast.

How The Projection Works

We combine recent signals with higher weight on short-term behavior:

- 65% from 30D momentum
- 35% from 90D momentum

That combined signal is mapped into:

- bullish if trend is meaningfully positive
- neutral if the signal is inconclusive
- bearish if trend is meaningfully negative

Then we widen or tighten the projected range using volatility:

- lower volatility = tighter range, higher confidence
- higher volatility = wider range, lower confidence

In simple terms:

the system checks whether the asset has been strong or weak recently, then adjusts confidence based on how unstable that asset usually is.

Data

The project uses Yahoo Finance historical data for 11 assets:

- AAPL
- AMZN
- GOOGL
- MSFT
- UDMY
- NXE
- SPY
- CDR.WA
- EH
- BTC-USD
- ETH-USD

Current local structure:

- data/raw/daily/: roughly 5 years of daily data
- data/raw/hourly/: recent hourly data for short-term momentum

Repository Structure

```
Financial-Agent/  
■■■ data/raw/daily/  
■■■ data/raw/hourly/  
■■■ notebooks/  
■ ■■■ eda_financial_agent.ipynb  
■ ■■■ frontend_payload_viewer.ipynb  
■■■ outputs/  
■ ■■■ frontend_payload.json  
■■■ scripts/  
■ ■■■ download_data.py  
■■■ src/  
■ ■■■ build_frontend_payload.py  
■■■ requirements.txt  
■■■ README.md
```

Notebooks

`notebooks/eda_financial_agent.ipynb`

Exploratory analysis notebook for:

- coverage and data quality
- normalized performance
- risk vs return
- drawdowns
- correlations
- recent momentum

`notebooks/frontend_payload_viewer.ipynb`

Visualization notebook for the generated backend payload.

Useful to inspect:

- portfolio summary
- per-asset metrics
- recommendations
- projections

- correlation matrix

Backend Payload

The backend analytical layer generates:

- [outputs/frontend_payload.json](/Users/pedrofs/EY_Challenge/Financial-Agent/outputs/frontend_payload.json)

This file is meant to be consumed by:

- the frontend
- the API layer
- the chatbot context layer

Top-level structure:

```
{
  "generated_at": "...",
  "portfolio_summary": {},
  "assets": [],
  "correlation_matrix": {}
}
```

Each asset contains:

```
{
  "ticker": "AAPL",
  "intelligence": {},
  "recommendation": {},
  "projection": {}
}
```

How To Run

1. Create a virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Download market data

```
python scripts/download_data.py
```

4. Build the frontend payload

```
python src/build_frontend_payload.py
```

5. Open the notebooks

```
jupyter notebook
```

Then use the Python (financial-agent) kernel in VS Code or Jupyter.

Recommendation Logic

The recommendation layer is rule-based and explainable.

In simple terms:

- buy: strong opportunity with acceptable risk
- hold: mixed or balanced signal
- reduce: weak opportunity with elevated risk

This was intentionally designed to be transparent and easy to defend in a business presentation.

Why This Approach

For a live investment-fund demo, the most important thing is not claiming magical price prediction.

The value of the platform is:

- faster portfolio review
- clearer risk visibility
- explainable recommendations
- better prioritization of opportunities

- decision support that can be consumed by both people and AI assistants

Output Summary

This project already produces a backend-ready analytical layer with:

- asset-level metrics
- ranking
- recommendation logic
- scenario projection
- frontend-consumable JSON

This makes it a strong base for:

- a dashboard
- a portfolio copilot
- a chatbot interface
- executive and technical demos